Multiversion Concurrency Control (MVCC) in DBMS

Presented by: Pallavi Singh |

Subject: DBMS

Introduction

What is DBMS?

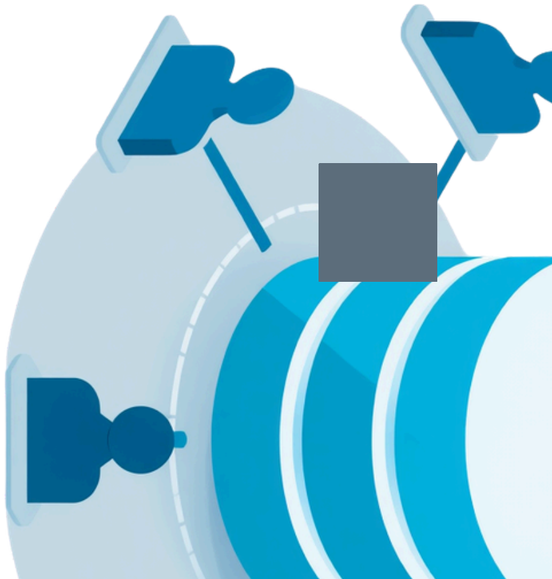
- A Database Management System (DBMS) is software that stores, retrieves, and manages data efficiently and securely.

What is Concurrency Control?

- It is a technique to manage simultaneous access to the database by multiple users or transactions without conflicts.

Why is it Important?

- Ensures data consistency, maintains integrity, and prevents errors when multiple operations occur at the same time.



Problems Without Concurrency Control

Without proper concurrency control, databases face critical issues:

- Dirty Read: Reading data from an uncommitted transaction that may be rolled back
- Lost Update: Two transactions overwrite each other's changes, losing data
- Uncommitted Data: Using temporary data that never gets finalized

Example:- Two users booking the same movie seat simultaneously - both see it available, both book it, but only one can actually get the seat!

Introduction to MVCC

(Multiversion Concurrency Control)

Multiversion Concurrency Control (MVCC) is a database technique that maintains multiple versions of data simultaneously.

- Readers don't block writers
- Writers don't block readers
- Each transaction sees a consistent snapshot of data
- Multiple versions coexist until no longer needed

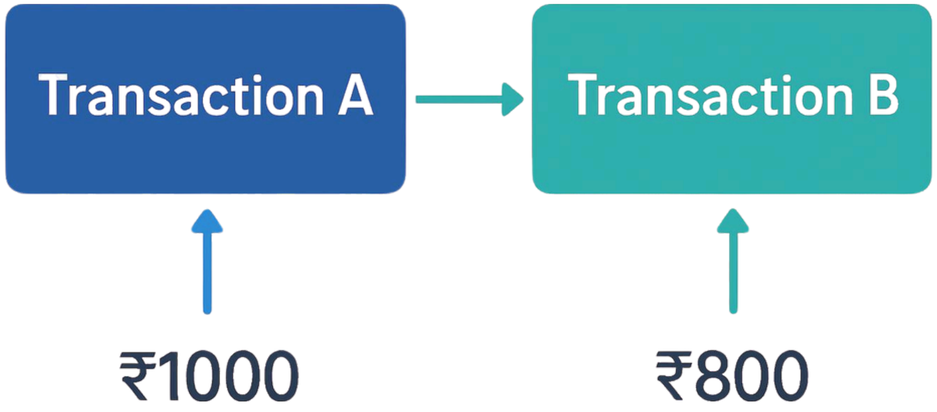
Notes:- MVCC solves the conflict between read and write operations by keeping old versions of data available. This means users can read data while others are updating it, without waiting or causing errors.

How MVCC Works

- **Snapshot Isolation:-** Each transaction sees a consistent snapshot of the database at its start time
- **Version Creation:-** When data is updated, a new version is created instead of overwriting the old one
- **Timestamps:-** Each version is tagged with timestamps to track when it was created and when it expires
- **Read Consistency:-** Readers access the appropriate version based on their transaction's timestamp
- **No Read Locks:-** Transactions can read data without blocking writers, improving concurrency

MVCC Example

- Transaction A reads account balance: ₹1000
- Transaction B updates balance to ₹800
- Transaction A still sees ₹1000 (old version preserved)
- Transaction B commits → new version (₹800) becomes current
- Result: Both transactions complete without conflicts!



Advantages of MVCC

- **No Read-Write Blocking:-** Readers never block writers and writers never block readers, significantly improving overall database performance and throughput.

- **Better Concurrency for Read-Heavy Workloads:-** Multiple transactions can read data simultaneously without waiting, making MVCC ideal for applications with high read operations.

- **Consistent Snapshots Without Locks:-** Each transaction sees a consistent snapshot of the database at a specific point in time, eliminating the need for expensive locking mechanisms.

Disadvantages

- **Storage Overhead:-** Multiple versions of data consume significant disk space, increasing storage costs and management complexity.
- **Garbage Collection Required:-** Old, obsolete versions must be periodically cleaned up, adding system overhead and requiring careful tuning.
- **Complex Implementation:-** MVCC systems are harder to design, debug, and maintain compared to simple locking mechanisms.

Real-World Usage

PostgreSQL uses MVCC as its default concurrency control mechanism, making it highly efficient for read-heavy workloads and complex queries.

Oracle, MySQL InnoDB, and Microsoft SQL Server also implement MVCC to handle high-performance transactions in enterprise applications.

Conclusion

MVCC enables efficient concurrent database access by maintaining multiple versions of data, allowing readers and writers to work simultaneously without blocking each other.

This technique is essential for modern database performance and consistency, powering major systems like PostgreSQL, MySQL InnoDB, and Oracle to handle thousands of concurrent transactions.

Thank You!